

SISTEMA MULTI-AGENTE INTELIGENTE

COPILOTO DE CONOCIMIENTO, ESTUDIO Y SEGUNDO CEREBRO

Manual Oficial de Usuario y Guía Técnica de Operación

Agent IA — Versión 1.0 (Arquitectura Híbrida)

Autor y Desarrollador:

Miguel Ángel Polo Castro

Entorno de Ejecución:

FastAPI + Streamlit + Ollama + Gemini

Integraciones Principales:

Notion (29 DBs Relacionales)

Obsidian (Vault Markdown)

ChromaDB (Memoria Vectorial)

Resumen del Documento

Esta guía detalla la configuración, despliegue, operación y buenas prácticas del sistema **Agent IA**. Se documenta la interacción en lenguaje natural con el orquestador **Hermes**, el funcionamiento de la flota de agentes especializados (**Curador**, **Plan**, **Estudio**, **Sync**), el sistema de protección y ofuscación de datos locales (**DataMasker**) y el modo híbrido conmutable entre modelos locales y en la nube.

Índice

1. Introducción y Visión General	2
1.1. ¿Qué es Agent IA?	2
1.2. Filosofía de Diseño: Human-in-the-Loop y Revisión Proactiva	2
2. Arquitectura del Sistema y Flota de Agentes	2
2.1. Descripción de los Agentes Especializados	2
3. Requisitos y Preparación del Entorno	3
3.1. Prerrequisitos de Software	3
3.2. Instalación de Dependencias con uv	3
3.3. Modelos Locales de Ollama	3
4. Configuración de Variables de Entorno (.env)	3
5. Puesta en Marcha y Servicios	4
5.1. Accesos Disponibles en el Navegador	4
6. Guía de Interacción y Casos de Uso	4
6.1. 1. Curaduría y Captura de Conocimiento (AgenteCurador)	5
6.2. 2. Planificación Estratégica de Proyectos y Metas (AgentePlan)	5
6.3. 3. Creación Rápida de Tareas (Fast-Path)	5
7. Sistema de Privacidad y Anonimización (DataMasker)	6
7.1. Mecanismo de Enmascaramiento y Restitución Local	6
7.2. Entidades Protegidas Automáticamente	6
8. Diagnóstico y Resolución de Problemas	6
9. Comandos de Referencia Rápida	6

1. Introducción y Visión General

1.1. ¿Qué es Agent IA?

Agent IA es un sistema multi-agente personal diseñado para operar como un copiloto proactivo de gestión de conocimiento, estudio y planificación estratégica. El sistema conecta y mantiene sincronizados los dos pilares del **Segundo Cerebro**:

1. **Notion:** Base relacional de alta estructuración con 29 bases de datos interconectadas (Objetivos, Proyectos, Tareas, Áreas, Recursos, etc.).
2. **Obsidian:** Vault local de notas en formato Markdown plano, garantizando portabilidad, velocidad y control total de archivos.

Toda la interacción con el usuario se realiza mediante **lenguaje natural** a través de **Hermes**, un agente orquestador central que clasifica intenciones, propone acciones y delega la ejecución en agentes especializados.

1.2. Filosofía de Diseño: Human-in-the-Loop y Revisión Proactiva

Agent IA opera bajo el principio estricto de **revisión humana**:

- **Propone, no destruye:** Las notas, metas o tareas nunca se escriben de forma destructiva sin confirmación previa si implican decisiones de categorización o estructuración compleja.
- **Aprobación conversacional:** El usuario puede confirmar propuestas mediante lenguaje natural ("sí", "dale", "hazlo", "perfecto") o a través de tarjetas interactivas con botones en la interfaz web.

2. Arquitectura del Sistema y Flota de Agentes

Agent IA está construido bajo una **Arquitectura Hexagonal (Ports & Adapters)**, lo que desacopla la lógica de negocio y agentes de los proveedores de inteligencia artificial (Ollama, Gemini) y de almacenamiento (Obsidian, Notion, ChromaDB).

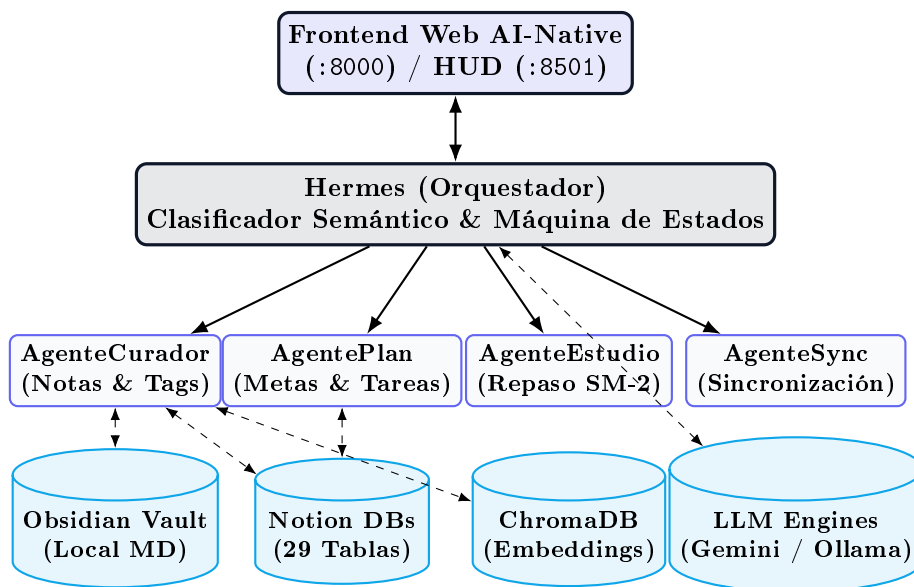


Figura 1: Diagrama de arquitectura hexagonal y flujo de delegación multi-agente.

2.1. Descripción de los Agentes Especializados

Tabla 1: Roles y responsabilidades de la flota de agentes.

Agente	Dominio	Responsabilidad Principal
Hermes	Orquestación	Analiza el prompt, gestiona memoria de sesión, rutea peticiones y administra el ciclo de confirmación activa.
AgenteCurador	Nota, Área	Extrae taxonomía (áreas, temas, tags), detecta duplicados vía búsqueda semántica y persiste en Notion + Obsidian.
AgentePlan	Tarea, Proyecto, Meta	Desglosa metas en planes estructurados (Objetivos → Proyectos → Tareas) y ejecuta inserciones relacionales.
AgenteEstudio	ItemEstudio	Administra tarjetas de estudio y calcula intervalos de repetición espaciada con el algoritmo SuperMemo SM-2.
AgenteSync	Integraciones	Vela por la consistencia de datos entre plataformas externas (Notion, Obsidian, Todoist, Calendar).

3. Requisitos y Preparación del Entorno

3.1. Prerrequisitos de Software

- **Sistema Operativo:** Windows 10/11 o Linux (Arch, Ubuntu, Debian).
- **Python:** Versión 3.14 o superior.
- **Gestor de Paquetes:** uv (Astral) para sincronización determinista de dependencias.
- **Ollama (Local):** Para generación de embeddings locales (`nomic-embed-text`) y fallback offline.
- **Google AI Studio API Key (Opcional pero Recomendado):** Para inferencia instantánea sub-segundo con Gemini Flash.

3.2. Instalación de Dependencias con uv

Desde la raíz del repositorio, ejecute:

```
# Sincronizar el entorno virtual completo
uv sync
```

3.3. Modelos Locales de Ollama

Asegúrese de descargar los pesos mínimos necesarios:

```
# Modelo ultraligero de embeddings (Memoria Vectorial ChromaDB)
ollama pull nomic-embed-text

# Modelo local ligero para fallback / offline
ollama pull llama3.2:3b
```

4. Configuración de Variables de Entorno (.env)

El archivo `.env` controla la conectividad, las claves criptográficas y el modo de operación del motor de inferencia.

Listing 1: Estructura completa de configuración en `.env`

```
# =====
```

```
# Agent IA -- Configuración Centralizada
# =====

# --- Integracion Notion ---
AGENT_NOTION_TOKEN=ntn_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
AGENT_NOTION_ROOT_PAGE_ID=6941a7f7-a5d2-463f-a718-6cb9030f7c8c

# --- Motor Local Ollama ---
AGENT_OLLAMA_BASE_URL=http://localhost:11434
AGENT_OLLAMA_MODEL=llama3.2:3b
AGENT_OLLAMA_MODEL_RAPIDO=llama3.2:3b
AGENT_OLLAMA_EMBED_MODEL=nomic-embed-text
AGENT_OLLAMA_KEEP_ALIVE=30m

# --- Almacenamiento Local (Obsidian y ChromaDB) ---
AGENT_OBSIDIAN_VAULT_PATH=C:\Users\migue\Agent_IA_Vault
AGENT_CHROMA_PERSIST_DIR=./data/chroma

# --- Backend LLM e Inferencia en la Nube ---
AGENT_LLM_BACKEND=hybrid # Opciones: hybrid | gemini | ollama
AGENT_GEMINI_API_KEY=AIzaSyXXXXXXXXXXXX # Clave de Google AI Studio
AGENT_GEMINI_MODEL=gemini-2.0-flash
AGENT_GEMINI_MODEL_PLAN=gemini-2.0-flash
AGENT_ENABLE_DATA_MASKING=true # Anonimizacion local de privacidad

# --- Puertos de Red ---
AGENT_API_HOST=0.0.0.0
AGENT_API_PORT=8000
AGENT_HUD_PORT=8501
```

Modos de Backend (AGENT_LLM_BACKEND)

- **hybrid (Recomendado):** Hermes, Curador y Plan utilizan Gemini Flash (< 800 ms), mientras que ChromaDB y los embeddings permanecen 100 % locales con nomic-embed-text.
- **ollama:** Inferencia 100 % local en tu máquina (sin conexión a internet).
- **gemini:** Todo el pipeline opera en la nube de Google.

5. Puesta en Marcha y Servicios

El sistema se compone de dos interfaces independientes servidas por FastAPI y Streamlit:

Listing 2: Terminal 1: API Gateway

```
# Iniciar Servidor FastAPI y Web
uv run agent-api
```

Listing 3: Terminal 2: HUD Streamlit (Opcional)

```
# Iniciar Panel Analitico Secundario
uv run agent-hud
```

5.1. Accesos Disponibles en el Navegador

- **Interfaz Web AI-Native Principal:** <http://localhost:8000/>
- **Documentación Interactiva Swagger / OpenAPI:** <http://localhost:8000/docs>
- **HUD de Métricas en Streamlit:** <http://localhost:8501/>

6. Guía de Interacción y Casos de Uso

6.1. 1. Curaduría y Captura de Conocimiento (AgenteCurador)

Para guardar una nota, recurso o apunte, basta con indicarlo a Hermes:

Ejemplo de Prompt para Curaduría

Usuario: *.Anota esto: Protocolo Raft resuelve consenso distribuido mediante elección de líder y replicación de log garantizando consistencia fuerte.*

Flujo de Ejecución:

1. Hermes detecta la intención y delega a **AgenteCurador**.
2. **AgenteCurador** genera el embedding del texto y busca notas similares en ChromaDB para prevenir duplicados.
3. El LLM categoriza la nota:
 - **Área sugerida:** Sistemas Distribuidos / Redes
 - **Tags:** consenso, raft, distribuidos
4. El sistema presenta una tarjeta interactiva de confirmación.
5. Al responder *"sí"* o presionar el botón [**Confirmar**]:
 - Se crea la página relacional en Notion con tags y área vinculada.
 - Se genera el archivo `Redes/Protocolo Raft.md` en el Vault de Obsidian.
 - Se indexa el ID de Notion en ChromaDB para búsquedas futuras.

6.2. 2. Planificación Estratégica de Proyectos y Metas (AgentePlan)

Ejemplo de Prompt para Planificación

Usuario: *"Quiero un plan estratégico para preparar mi examen final de Redes de Computadores en 3 semanas."*

Resultado Estructurado que Genera el Sistema:

- **Objetivo Principal:** *Aprobar Examen Final de Redes de Computadores (Área: Universidad).*
- **Proyecto 1: Dominio de Capa Física y Enlace**
 - [] Resolver laboratorio de modulación y sincronismo.
 - [] Elaborar tabla comparativa de tramas UART vs SPI.
- **Proyecto 2: Enrutamiento y Capa de Red**
 - [] Configurar topología OSPF y BGP en simulador.
 - [] Practicar ejercicios de subnetting VLSM.

Al confirmar, el sistema inserta automáticamente las entidades en cascada en las bases de datos de Notion (**Objetivos** → **Proyectos** → **Tareas**) estableciendo las relaciones bidireccionales.

6.3. 3. Creación Rápida de Tareas (Fast-Path)

Si solo necesitas agregar una tarea atómica sin desglose estratégico:

Comando Rápido

Usuario: *.agrega una tarea: Enviar informe de laboratorio a David Herrera antes de las 6pm"*

El sistema omite el procesamiento de LLM, ejecuta la inserción directa en la base de datos de **Tareas** de Notion y responde en menos de 300 ms.

7. Sistema de Privacidad y Anonimización (DataMasker)

Para garantizar la confidencialidad de la información cuando se utiliza el backend de Gemini Flash en la nube, el sistema incorpora el middleware **DataMasker** (`sanitizer.py`).

7.1. Mecanismo de Enmascaramiento y Restitución Local

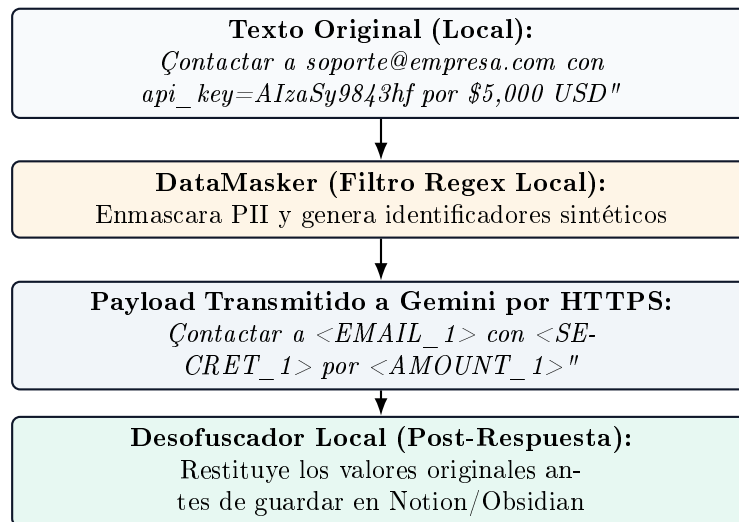


Figura 2: Flujo de sanitización local y protección de privacidad.

7.2. Entidades Protegidas Automáticamente

- **Secretos y Credenciales:** `api_key`, `token`, `password`, contraseña.
- **Direcciones de Correo:** Cualquier formato estándar `user@domain.ext`.
- **Números Telefónicos:** Secuencias locales e internacionales de 7 a 15 dígitos.
- **Montos Financieros:** Cifras con símbolos de moneda (\$, €, USD, COP, EUR, etc.).

8. Diagnóstico y Resolución de Problemas

9. Comandos de Referencia Rápida

Tabla 2: Guía de diagnóstico de errores comunes.

Síntoma	Causa Probable	Solución
Error 429 Too Many Requests	Límite de peticiones de Google AI Studio alcanzado.	El sistema activa el fallback local automáticamente. Si persiste, espere 60 segundos o configure una API key con facturación.
Respuestas tardan > 30 s	El backend está en modo ollama ejecutando modelos pesados en CPU.	Cambie en <code>.env</code> a <code>AGENT_LLM_BACKEND=hybrid</code> o baje el modelo local a <code>llama3.2:3b</code> .
Error de conexión en Notion	Token de Notion expirado o página raíz no compartida con la integración.	Verifique en Notion que la integración Agent IA tenga permisos de lectura/escritura en la página raíz.
ChromaDB no inicia	Permisos de escritura bloqueados en la carpeta <code>./data/chroma</code> .	Asegúrese de que el usuario tenga permisos de escritura en la carpeta del proyecto.

Tabla 3: Comandos esenciales de la CLI del proyecto.

Comando	Acción
<code>uv run agent-api</code>	Inicia el API Gateway de FastAPI y la interfaz web en <code>http://localhost:8000</code> .
<code>uv run agent-hud</code>	Inicia el HUD de monitoreo en Streamlit en <code>http://localhost:8501</code> .
<code>uv run pytest -v tests/</code>	Ejecuta la suite completa de 46 pruebas unitarias y de integración.
<code>python scripts/sync_notion_map.py</code>	Re-escanea las 29 bases de datos de Notion y regenera la caché local.
<code>ollama list</code>	Muestra los modelos de inteligencia artificial instalados localmente.